

Titulación: Grado en Ingeniería Informática, Ingeniería en Sistemas de Información e InfoAde
Curso: 2024-2025. Convocatoria Ordinaria de Junio
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 2: Carga Masiva de Datos, Procesamiento y Optimización de Consultas

ALUMNO 1:

Nombre y Apellidos: Gonzalo López Menor

DNI: 03210671D

ALUMNO 2:

Nombre y Apellidos: Juan Cartagena Castro

DNI: 09101498F

Fecha: 3/04/2025

Profesor Responsable: Maria Teresa Villalba

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se calificará la asignatura como Suspenso – Cero.

Plazos

Tarea en Laboratorio: Semana 10 de marzo, Semana 17 de marzo, Semana 24 de marzo, semana 31 de marzo y 7 de abril.

Entrega de práctica: Día 20 de abril, domingo a las 23:59. Aula Virtual

Documento a entregar: Este mismo fichero con las respuestas a las cuestiones planteadas, el programa que genera los datos de carga de la base de datos, los ficheros de log de PostgreSQL de esta práctica. No se piden los ficheros de los datos en bruto de la base de datos. Se entregará en un ZIP comprimido llamado: **DNI'sdelosAlumnos_PECL2.zip**

AMBOS ALUMNOS DEBEN ENTREGAR EL FICHERO EN LA PLATAFORMA.

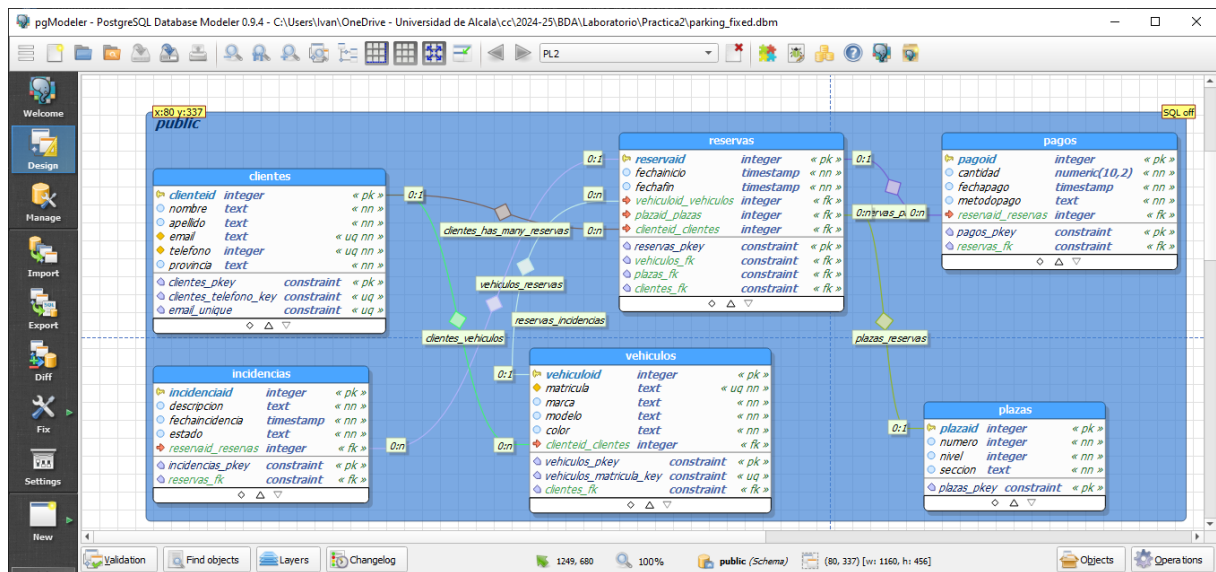
Introducción

El contenido de esta práctica versa sobre la monitorización de la base de datos, manipulación de datos, técnicas para una correcta gestión de los mismos, así como tareas de mantenimiento relacionadas con el acceso y gestión de los datos. También se trata el tema de procesamiento y optimización de consultas realizadas por PostgreSQL (17). Se analizará PostgreSQL en el proceso de carga masiva y optimización de consultas.

En general, la monitorización de la base de datos es de vital importancia para la correcta implantación de una base de datos, y se suele utilizar en distintos entornos:

- Depuración de aplicaciones: cuando se desarrollan aplicaciones empresariales no se suele acceder a la base de datos a bajo nivel, sino que se utilizan librerías de alto nivel y mapeadores ORM (Hibernate, Spring Data, MyBatis...) que se encargan de crear y ejecutar consultas para que el programador pueda realizar su trabajo más rápido. El problema en estos entornos está en que se pierde el control de qué están haciendo las librerías en la base de datos, cuántas consultas ejecutan, y con qué parámetros, por lo que la monitorización en estos entornos es vital para saber qué consultas se están realizando y poder optimizar la base de datos y los programas en función de los resultados obtenidos.
- Entornos de prueba y test de rendimiento: cuando una base de datos ha sido diseñada y se le cargan datos de prueba, una de las primeras tareas a realizar es probar que todos los datos que almacenan son consistentes y que las estructuras de datos dan un rendimiento adecuado a la carga esperada. Para ello se desarrollan programas que simulen la ejecución de aquellas consultas que se consideren de interés para evaluar el tiempo que le lleva a la base de datos devolver los resultados, de cara a buscar optimizaciones, tanto en la estructura de la base de datos como en las propias consultas a realizar.
- Monitorización pasiva/activa en producción: una vez la base de datos ha superado las pruebas y entra en producción, el principal trabajo del administrador de base de datos es mantener la monitorización pasiva de la base de datos. Mediante esta monitorización el administrador verifica que los parámetros de operación de la base de datos se mantienen dentro de lo esperado (pasivo), y en caso de que algún parámetro salga de estos parámetros ejecuta acciones correctoras (reactivo). Así mismo, el administrador puede evaluar nuevas maneras de acceso para mejorar aquellos procesos y tiempos de ejecución que, pese a estar dentro de los parámetros, muestren una desviación tal que puedan suponer un problema en el futuro (activo).

Para la realización de esta práctica será necesario generar una muestra de datos de cierta índole en cuanto a su volumen de datos. Para ello se generarán, dependiendo del modelo de datos suministrado, para una base de datos denominada **TELPARK**. Básicamente la base de datos guarda las reservas de los clientes que se realizan en el parking junto con su método de pago, los vehículos y las incidencias que se pueden producir. Se suministra el modelo relacional construido en **pgmodeler**.



Los datos referidos al año 2024 que hay que generar deben de ser los siguientes:

- Existen 3.000.000 clientes registrados en la APP de Telpark. Las provincias deben de ser asignada aleatoriamente de entre todas las provincias españolas.
- Existen 5.000.000 vehículos dados de alta. Se deben asignar los vehículos a los propietarios de manera aleatoria. Existen 500 marcas de automóviles diferentes donde cada una tiene entre 5 y 20 modelos, con colores entre 5 y 15 (el negro debe ser uno de ellos). La asignación de los valores de cada campo se debe de hacer de manera aleatoria.
- Existen 200.000 plazas de parking de la empresa Telpark. La planta debe ser un número aleatorio entre -10 y 0, y la sección un valor aleatorio comprendido entre las letras A-F.
- Se han producido 40.000.000 de reservas de plazas de parking por la APP. La fecha de inicio y final debe tener el día, mes, y hora generados de manera aleatorio, con una duración de estancia comprendida entre 1 día y 10 días.
- Cada reserva se ha pagado por un método de pago que puede ser efectivo, tarjeta de crédito, tarjeta de débito, PayPal, Bizzum o transferencia. La cantidad de la reserva debe ser el número de horas multiplicado por 3€. La fecha de pago debe ser anterior al comienzo de la estancia en el Parking.
- Se han generado 4.000.000 de incidencias generadas aleatoriamente de entre todas las reservas, donde el estado puede ser nueva, abierta, en proceso, resuelta y cerrada. Todas las asignaciones se hacen de manera aleatoria.

Actividades y Cuestiones

Cuestión 0: Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezca el DNI de los alumnos que realizan la

práctica (ambos), el nombre del host con su puerto, y la fecha y hora de la operación que se ha realizado. **Se ha de configurar también el servidor para que no use el procesamiento paralelo de consultas.**

Nos vamos a la ruta correspondiente donde esté el archivo postgres.conf
En este caso: C:\Program Files\PostgreSQL\16\data\log
Y modificamos el parámetro log_statement estableciéndolo en ' mod '
Después hay que modificar el prefijo de la línea de registro line_prefix incluyendo el dni, nombre host y el puerto. Esto se hace mirando lo que nos pone el archivo .conf en cuanto a instrucciones.
La duración de consulta la desactivamos con log_duration = off
En la sentencia max_parallel_workers_per_gate lo establecemos en 0 para que no use el procesamiento paralelo de consultas. (Esto lo vemos en la documentación de postgres, en la sección Resource Consumption)

Cuestión 1: ¿Tiene el servidor postgres un recolector de estadísticas sobre los datos de cada campo de las tablas? Si es así, ¿Qué tipos de estadísticas se recolectan y dónde se guardan?

Según la documentación, postgres si emplea un recolector de estadísticas sobre los datos de cada campo de las tablas.(Mediante comando ANALYZE). Tiene estimación de filas y estadísticas multivariantes.
Para cada columna recopila la proporción de entradas nulas en la columna
El tamaño medio de los datos almacenados en la columna.
El número de valores distintos(n_distinct)
Los valores más comunes(MCV) y sus frecuencias.
Un histograma y la correlación(correlation) entre el orden físico de las filas y el orden lógico de los datos.
Todas estas estadísticas se guardan en la tabla de pg_statistic, la cual podemos consultar para comprobar las estadísticas mencionadas.

Cuestión 2: Crear una nueva base de datos llamada **investigar** y que tenga las siguientes tablas con los siguientes campos y características:

- investigadores(codigo_investigador tipo numeric PRIMARY KEY, nombre tipo text, apellidos tipo text, salario tipo numeric)
- contratos(codigo_contrato tipo numeric PRIMARY KEY, nombre tipo text, entidad tipo text, coste tipo numeric)
- investigadores_contratos(codigo_investigador tipo numeric que sea FOREIGN KEY del campo codigo_investigador de la tabla investigadores con restricciones de tipo RESTRICT en sus operaciones, codigo_contrato tipo numeric que sea FOREIGN KEY del campo codigo_contrato de la tabla contratos con restricciones de tipo RESTRICT en sus operaciones, horas de tipo numeric. La PRIMARY KEY debe ser compuesta de codigo_investigador y codigo_contrato.

Se pide:

- Indicar el proceso seguido para generar esta base de datos.
- Cargar la información del fichero datos_empleados.csv, datos_proyectos.csv y datos_trabaja_proyectos.csv en dichas tablas de tal manera que sea lo más eficiente posible.
- Indicar los tiempos de carga.

Antes de nada vamos a habilitar los tiempos de carga con \timing on
Creamos la base de datos investigar CREATE DATABASE pl2avanzadas;
Luego, según la documentación de postgres, deberemos de crear las tablas sin restricciones para después del copy agregarlas, haciendo que esta forma sea la más eficiente, y tardando 20 minutos menos en copiar datos que con las restricciones establecidas ya.
Creamos la tabla investigadores CREATE TABLE investigadores (codigo_investigador NUMERIC, nombre TEXT, apellidos TEXT, salario NUMERIC);
Creamos la tabla contratos: CREATE TABLE contratos (codigo_contrato NUMERIC, nombre TEXT, entidad TEXT, coste NUMERIC);
La tabla investigadores_contratos:
CREATE TABLE investigadores_contratos (codigo_investigador NUMERIC, codigo_contrato NUMERIC, horas NUMERIC);
Según la documentación de postgres, empleamos COPY y no empleamos CSV HEADER. Esto lo hacemos para que no exista una validación constante de la integridad referencial y se reduzca al mínimo los tiempos de carga. Además se podrían realizar insert, pero copy es la mejor opción para la carga masiva de datos.
\COPY investigadores (codigo_investigador, nombre, apellidos, salario)
FROM 'datos_investigadores.csv' DELIMITER ',';
El primer COPY ha tardado 16809,118 ms
\COPY contratos (codigo_contrato, nombre, entidad, coste) FROM 'datos_contratos.csv'
DELIMITER ',';
El segundo COPY ha tardado 962,689 ms
\COPY investigadores_contratos (codigo_investigador, codigo_contrato, horas)
FROM 'datos_investigadores_contratos.csv' DELIMITER ',';
El tercer COPY ha tardado
Duración: 42291,586 ms (00:42,292)
Luego agregamos las restricciones con ALTER TABLE y ya lo tendríamos.

Cuestión 3: Mostrar las estadísticas obtenidas en este momento para cada tabla. ¿Qué se almacena? ¿Son correctas? Si no son correctas, ¿cómo se pueden actualizar?

Vamos a emplear esta consulta para poder tener una salida más limpia: `SELECT relname, n_live_tup, n_tup_ins, n_tup_upd, n_tup_del FROM pg_stat_user_tables;`

Vemos que se almacena el numero de tuplas vivas, el nombre de la tabla, el numero de tuplas insertadas, el numero de tulas actualizadas, el numero de tuplas eliminadas entre otras estadísticas, ya que hemos seleccionado unas precisas. Básicamente información acerca de las tablas que existen.

Se puede observar que no es correcto el numero de tuplas vivas ya que acabamos de copiar los datos hace nada, por lo que vamos a emplear un `ANALYZE` para actualizar las estadísticas y conseguirlas con más precisión.

Cuestión 4: Aplicar el comando `EXPLAIN` a una consulta que obtenga la información de los investigadores con salario de menos de 24.000 euros. ¿Son correctos los resultados del comando `EXPLAIN`? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de las tablas con postgres.

Aplicamos la siguiente consulta empleando el comando `EXPLAIN`: `EXPLAIN SELECT * FROM investigadores WHERE salario < 24000;`

Esto nos otorga la salida

```
pl2avanzadas=# EXPLAIN SELECT * FROM investigadores WHERE salario < 24000;
               QUERY PLAN
```

Seq Scan on investigadores (cost=0.00..65540.00 rows=343685 width=41)

Filter: (salario < '24000'::numeric)

(2 filas)

Sacamos la LR de cada atributo desde la documentacion de postgresql. (data types)

Para calcular el factor de bloque teóricamente, hemos tenido en cuenta que el nombre es un tipo `TEXT` y mirando en la documentación de postgres, sabemos que cada carácter ocupa 1 byte, por lo que hemos tenido en cuenta el cálculo con el valor mas grande del campo nombre.

En el análisis teórico hemos estimado un coste de lectura secuencial de 125.000 bloques para la tabla investigadores. Sin embargo, el comando `EXPLAIN` nos devuelve un coste de 0.00..65540.00. Esta diferencia puede ser porque PostgreSQL utiliza unidades de coste relativas que consideran no solo los bloques leídos, sino también operaciones CPU y parámetros de configuración como `seq_page_cost` y `cpu_tuple_cost` además de que considera el plan de ejecución más barato a nivel de coste.

Cuestión 5: Aplicar el comando EXPLAIN a una consulta que obtenga el número de contratos en los cuales el investigador trabaja 8 horas y tiene un salario de 20.000 euros. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de las tablas con postgres.

Empleamos la consulta correspondiente con el comando explain: EXPLAIN SELECT COUNT(*) FROM investigadores i JOIN investigadores_contratos ic ON i.codigo_investigador = ic.codigo_investigador WHERE ic.horas = 8 AND i.salario = 20000;

Vemos que postgres lo que hace es asumir el coste de la agregación, usando el nested loop, búsqueda secuencial sobre investigadores y además usar el index scan en investigadores_contratos(pkkey). Los cálculos teóricos(ppt) nos llevan a la conclusión de que postgres emplea los índices creados automáticamente pkkey para evitar realizar una lectura completamente secuencial. El resultado teórico secuencial sería de un coste del join 0, mas el coste de las lecturas secuenciales(o binarias teniendo en cuenta si está ordenada la tabla correctamente), en el caso de secuenciales sería: 605.043 bloques. En la práctica, según la documentación de postgres(capítulo 14.1, using explain) postgres ha elegido una Nested Loop con un Index Scan en investigadores_contratos, aprovechando el índice primario (pkkey) para reducir el número de bloques leídos. A diferencia de los cálculos teóricos, postgres reduce los costes gracias a sus índices automáticos. Por tanto, el plan elegido tiene un coste mucho menor que el teórico, demostrando la ventaja de los índices en consultas filtradas y joins por clave.

Cuestión 6: Aplicar el comando EXPLAIN a una consulta que obtenga la información del nombre de los contratos y entidad que tienen un coste mayor de 10000 y menor que 15000, y tienen investigadores de salario mayor de 16000 euros y trabajan 5 horas en ellos. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de las tablas con postgres.

Empleamos la consulta con el comando explain EXPLAIN SELECT c.nombre, c.entidad FROM contratos c JOIN investigadores_contratos ic ON c.codigo_contrato = ic.codigo_contrato JOIN investigadores i ON ic.codigo_investigador = i.codigo_investigador WHERE c.coste > 10000 AND c.coste < 15000 AND i.salario > 16000 AND ic.horas=5;

PostgreSQL ha elegido un plan que empieza con un Hash Join entre las tablas contratos e investigadores_contratos, uniendo por claves foráneas. Además, se ve que utiliza una búsqueda secuencial en investigadores_contratos, una búsqueda secuencial en contratos y emplea el index scan sobre investigadores_pkkey. Después, el resultado se une con la tabla investigadores mediante un Nested Loop, donde sí se utiliza un Index Scan sobre la clave primaria investigadores_pkkey. Esto permite aplicar el filtro salario > 16000 de forma eficiente sin tener que recorrer toda la tabla. Teóricamente, vemos que se utilizan una barbaridad más de bloques suponiendo que únicamente podemos emplear para los join BAB, y que el coste de lectura inicial de las ramas es binario. Con esto, se puede llegar a ver que los cálculos teóricos no siempre son exactos ni aproximados, y que postgres emplea índices pk's, hash join y todo lo necesario para que la consulta sea lo más eficiente posible y consiga ahorrarse todo el coste que pueda.

Cuestión 7: Generar los datos solicitados al comienzo de la práctica para la base de datos **TELPARK** creando un programa para tal fin que deberá de estar en un único fichero y comentado. Pegar el código del fichero en el cuadro de texto que se adjunta a continuación.

No entra todo el código, pero:

```
import random
import csv
from datetime import datetime, timedelta

# Configuración
random.seed(42) # Para reproducibilidad
NUM_CLIENTES = 3_000_000
NUM_VEHICULOS = 5_000_000
NUM_PLAZAS = 200_000
NUM_RESERVAS = 40_000_000
NUM_INCIDENCIAS = 4_000_000

# Datos de referencia
PROVINCIAS_ESPANOLAS = ['Álava', 'Albacete', 'Alicante', 'Almería', 'Asturias', 'Ávila', 'Badajoz',
                        'Balears', 'Barcelona', 'Burgos', 'Cáceres', 'Cádiz', 'Cantabria',
                        'Castellón', 'Ciudad Real', 'Córdoba', 'Cuenca', 'Gerona', 'Granada',
                        'Guadalajara', 'Guipúzcoa', 'Huelva', 'Huesca', 'Jaén', 'La Coruña',
                        'La Rioja', 'Las Palmas', 'León', 'Lérida', 'Lugo', 'Madrid', 'Málaga',
                        'Murcia', 'Navarra', 'Orense', 'Palencia', 'Pontevedra', 'Salamanca',
                        'Santa Cruz de Tenerife', 'Segovia', 'Sevilla', 'Soria', 'Tarragona',
```

Cuestión 8: Realizar la carga masiva de los datos generados en la cuestión 7 en la base de datos **TELPARK**. Indicar el proceso seguido y el orden de carga de las tablas, explicando el porqué de dicho orden; **y asegurando la consistencia e integridad de los datos cargados**. Comparar los tiempos en las tablas implicadas y explicar a qué es debida la diferencia.

Tabla	Tiempo (seg)
Clientes	16,977
Vehiculos	14,447
Plazas_Parking	0,283
Reservas	504
pagos	330
Incidencias	41,820

Lo primero de todo es crear las tablas sin ningún tipo de restricciones para acelerar el proceso de copy. Para mantener la integridad referencial, es necesario cargar primero las tablas que no dependen de otras. Cargamos primero la tabla clientes que no depende de nadie y luego la tabla vehículos que depende de clientes. Así sucesivamente con plazas, reservas (que depende de clientes, plazas y vehículos), pagos (dependiente de reservas) e incidencias (dependiente de reservas).

Los datos se cargan concretamente de esta forma para asegurar la integridad referencial en todo momento, es decir que no falte ningún dato en ninguna tabla en ningún momento, por lo que por ejemplo, no podríamos generar primero los datos de la tabla reservas, ya que esta tabla requiere que exista previamente un cliente, un vehículo y una plaza.

Tras todo esto, creamos las restricciones correspondientes al modelo que nos han otorgado de telpark con alter table y observamos las comparaciones de los tiempos de los copy's de cada tabla.

Como podemos ver, la tabla reservas ha tardado un aproximado de 9 minutos en copiar todos los datos, algo lógico debido a su gran cantidad de registros (40.000.000), que comparado con las demás tablas, la carga es mucho menor.

A partir de este momento en adelante, se deben de realizar las siguientes cuestiones con la base de datos que tiene la integridad referencial activada. Es obligatorio y queda prohibido cambiar la integridad referencial de la base de datos.

Cuestión 9: Realizar una consulta SQL que muestre el “porcentaje de clientes que cumplen que son de la comunidad de Andalucía realizando una reserva durante los meses de verano, pagando una cantidad de más de 150 €, estacionando sus vehículos por debajo de la planta -4, y que no hayan tenido incidencias cerradas con sus vehículos de color negro”.

Obtener el plan de ejecución con el resultado del comando EXPLAIN. Explicar la información obtenida en el plan de ejecución de PostgreSQL. Comparar el árbol obtenido por nosotros al traducir la consulta original al álgebra relacional y el que obtiene PostgreSQL. Comentar las posibles diferencias entre ambos árboles.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
SELECT (COUNT(DISTINCT c.clienteid) * 100.0 / (SELECT COUNT(*) FROM clientes)) AS
porcentaje_clientes_andalucia FROM clientes c JOIN reservas r ON c.clienteid = r.clienteid_clientes
JOIN vehiculos v ON r.vehiculoid_vehiculos = v.vehiculoid JOIN plazas p ON r.plazaid_plazas =
p.plazaid JOIN pagos p2 ON r.reservaid = p2.reservaid_reservas LEFT JOIN incidencias i ON
r.reservaid = i.reservaid_reservas
WHERE c.provincia IN ('Almería', 'Cádiz', 'Córdoba', 'Granada', 'Huelva', 'Jaén', 'Málaga', 'Sevilla')
AND EXTRACT(MONTH FROM r.fechainicio) IN (6, 7, 8) AND p2.cantidad > 150 AND p.numero
< -4 AND (i.estado != 'cerrada' OR i.estado IS NULL) AND v.color != 'negro';
porcentaje_clientes_andalucia
```

4.418266666666667

(1 fila)

Duración: 194209,058ms(03:14,209)

Resultado comando EXPLAIN

QUERY PLAN

Aggregate (cost=2288810.14..2288810.16 rows=1 width=32)

InitPlan 1 (returns \$0)

-> Aggregate (cost=84444.00..84444.01 rows=1 width=8)

-> Seq Scan on clientes (cost=0.00..76944.00 rows=3000000 width=0)

-> Sort (cost=2204307.75..2204336.94 rows=11677 width=4)

Sort Key: c.clienteid

-> Nested Loop (cost=1212160.63..2203518.89 rows=11677 width=4)

-> Hash Join (cost=1212160.19..2160215.43 rows=12485 width=8)

Hash Cond: (p2.reservaid_reservas = r.reservaid)

-> Seq Scan on pagos p2 (cost=0.00..827673.80 rows=32068423 width=4)

Filter: (cantidad > '150'::numeric)

-> Hash (cost=1211965.53..1211965.53 rows=15573 width=12)

-> Hash Right Join (cost=1104813.17..1211965.53 rows=15573 width=12)

Hash Cond: (i.reservaid_reservas = r.reservaid)

Filter: ((i.estado <> 'cerrada'::text) OR (i.estado IS NULL))

-> Seq Scan on incidencias i (cost=0.00..92128.00 rows=4000000 width=12)

-> Hash (cost=1104569.53..1104569.53 rows=19491 width=12)

-> Hash Join (cost=114854.25..1104569.53 rows=19491 width=12)

Hash Cond: (r.clienteid_clientes = c.clienteid)

-> Hash Join (cost=4988.00..989937.45 rows=328315 width=12)

Hash Cond: (r.plazaid_plazas = p.plazaid)

-> Seq Scan on reservas r (cost=0.00..983374.45 rows=599991 width=16)

Filter: (EXTRACT(month FROM fechainicio) = ANY ('{6,7,8}'::numeric

{}))

-> Hash (cost=3620.00..3620.00 rows=109440 width=4)

-> Seq Scan on plazas p (cost=0.00..3620.00 rows=109440 width=4)

Filter: (numero < '-4'::integer)

-> Hash (cost=106944.00..106944.00 rows=178100 width=4)

-> Seq Scan on clientes c (cost=0.00..106944.00 rows=178100 width=4)

Filter: (provincia = ANY ('

{Almería,Cádiz,Córdoba,Granada,Huelva,Jaén,Málaga,Sevilla}'::text[]))

-> Index Scan using pk_vehiculos on vehiculos v (cost=0.43..3.47 rows=1 width=4)

Index Cond: (vehiculoid = r.vehiculoid_vehiculos)

Filter: (color <> 'negro'::text)

(32filas)

Comentarios y explicaciones.

Lo que emplea Postgresql es un Hash Join entre las tablas vehiculos e incidencias, unidas por la clave foránea del vehículo. Este hash join permite unir de forma eficiente las incidencias relacionadas con los vehículos, aplicando desde el principio el filtro gracias a un Index Scan sobre la tabla incidencias. Posteriormente, se realiza un Nested Loop entre los vehículos filtrados por planta y el conjunto anterior de vehículos con incidencias cerradas. Esta operación permite excluir a los vehículos que han tenido dichas incidencias, utilizando un Index Scan sobre vehiculos y aplicando el filtro., En el siguiente paso, se realiza un Join con la tabla reservas, uniendo por la clave del vehículo (vehiculoid_vehiculos). PostgreSQL selecciona un Hash Join, ya que las reservas también están siendo filtradas por la condición temporal sobre los meses de verano, que puede aprovechar los índices sobre fecha_inicio. Seguidamente, la tabla pagos se une utilizando un Nested Loop Join, aplicando el filtro cantidad > 150, y posiblemente aprovechando un Bitmap Index Scan sobre los índices de las primary key's. A continuación, se enlaza con la tabla clientes, que es la que permite acceder a la condición regional Este último join puede utilizar un Index Scan sobre la clave primaria de clientes para aplicar de forma eficaz el filtro geográfico. Todo esto no tiene apenas relacion con la cantidad que obtenemos de la teoría debido a la falta de información que postgre si emplea y nosotros no.

Cuestión 10: Usando PostgreSQL, y a raíz de los resultados de la cuestión anterior, ¿qué acciones realizaría para mejorar el rendimiento de la consulta sin cambiar el código SQL y por qué? Obtener la información pedida de la cuestión 9 y explicar los resultados. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Comentar los resultados obtenidos y comparar con la cuestión anterior.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
(Empleamos los CREATE INDEX necesarios...)
SELECT (COUNT(DISTINCT c.clienteid) * 100.0 / (SELECT COUNT(*) FROM clientes)) AS
porcentaje_clientes_andalucia FROM clientes c JOIN reservas r ON c.clienteid = r.clienteid_clientes
JOIN vehiculos v ON r.vehiculoid_vehiculos = v.vehiculoid JOIN plazas p ON r.plazaid_plazas =
p.plazaid JOIN pagos p2 ON r.reservaid = p2.reservaid_reservas LEFT JOIN incidencias i ON
r.reservaid = i.reservaid_reservas
WHERE c.provincia IN ('Almería', 'Cádiz', 'Córdoba', 'Granada', 'Huelva', 'Jaén', 'Málaga', 'Sevilla')
AND EXTRACT(MONTH FROM r.fechainicio) IN (6, 7, 8) AND p2.cantidad > 150 AND p.numero
< -4 AND (i.estado != 'cerrada' OR i.estado IS NULL) AND v.color != 'negro';
porcentaje_clientes_andalucia
-----
4.4182666666666667
Duración: 131984,519 ms (02:11,985)
```

Resultado comando EXPLAIN

```
QUERY PLAN
Aggregate (cost=2212803.03..2212803.05 rows=1 width=32)
  InitPlan 1 (returns $0)
    -> Aggregate (cost=62836.41..62836.42 rows=1 width=8)
      -> Index Only Scan using idx_clientes_provincia on clientes (cost=0.43..55336.41
rows=3000000 width=0)
    -> Sort (cost=2149908.22..2149937.41 rows=11677 width=4)
      Sort Key: c.clienteid
    -> Nested Loop (cost=1157740.00..2149119.36 rows=11677 width=4)
      -> Hash Join (cost=1157739.57..2105812.42 rows=12486 width=8)
        Hash Cond: (p2.reservaid_reservas = r.reservaid)
        -> Seq Scan on pagos p2 (cost=0.00..827688.00 rows=32069333 width=4)
          Filter: (cantidad > '150'::numeric)
        -> Hash (cost=1157544.91..1157544.91 rows=15573 width=12)
          -> Hash Right Join (cost=1050392.54..1157544.91 rows=15573 width=12)
```

Comentarios y explicaciones.

La mejor opción para mejorar el rendimiento de la consulta sin cambiar código lo hemos visto muchas veces en anteriores trabajos y demás, es usando índices que ayudan a facilitar las búsquedas. Vamos a crear índices en clientes_provincia, reservas_fecha_inicio, pagos_cantidad, plazas_numero y vehiculos_color. Lo que hace postgresql es emplear sucesivos nested loop, hash join y lecturas secuenciales que hacen que los costes sean muy elevados, pero también emplea índices que hemos creado sobre los atributos en los que hace join's para que el coste y el tiempo sea bastante menor y conseguir así una consulta mucho más optimizada. La comparación con la cuestión anterior se ve simplemente por el coste y el tiempo, se puede observar que los índices ayudan a postgresql para poder decidir qué plan de ejecución emplear y para aumentar el rendimiento en la búsqueda.

Cuestión 11: Usando PostgreSQL, borre el 30% de los clientes almacenados de manera aleatoria y todos sus datos relacionados de la manera más eficiente posible, ¿cuál ha sido el proceso seguido? ¿Y el tiempo empleado en el borrado? **Prohibido modificar la Integridad Referencial.**

Para eliminar los datos de la manera más eficiente posible, aleatoria y manteniendo la integridad referencial, creamos una tabla temporal con el 30% de manera aleatoria con: `CREATE TEMP TABLE clientes_a_eliminar AS SELECT clienteid FROM clientes ORDER BY RANDOM() LIMIT( SELECT COUNT(*) * 0.3 FROM clientes);`
Con todo, creamos un índice sobre la tabla temporal para que (tras varios días) podamos optimizar su rendimiento a la hora de borrar. `CREATE INDEX idx_clientes_borrar_clienteid ON clientes_a_borrar (clienteid);` Eliminamos los datos de manera que se mantenga la integridad referencial, primero las incidencias, luego pagos, reservas, vehiculos y por último los clientes. Empleamos `DELETE FROM... USING` para las cantidades masivas de datos y en vez de hacer consultas anidadas que conviertan la consulta en un uso de hash y lecturas secuenciales que obliguen a materializar todos los registros. Por ejemplo: `DELETE FROM reservas USING vehiculos v, clientes_a_borrar cab WHERE reservas.vehiculoid_vehiculos = v.vehiculoid AND v.clienteid_clientes = cab.clienteid;`
El borrado de incidencias ha sido de (00:26,355) mientras que el de pagos han sido 18 minutos (18:00,423) + (07:04,766). El borrado de reservas ha empleado los índices correctamente y ha tardado 22 minutos y medio (22:23,441) + 5 minutos (05:02,253) + (11:24,129) comparados con el tiempo que tardaba anteriormente (No terminaba el borrado). Aquí nos dimos cuenta de que también era necesario borrar las reservas de un vehículo que podría haber hecho otro cliente cualquiera pero que esté sobre ese vehículo (nos saltó la restricción), por lo que corregimos los deletes. El borrado de vehículos tardó (02:26,104) y el final de clientes: (01:17,155).
Estos tiempos no son nada comparados con los que daban antes (horas y horas), por lo que se ve que el uso de índices ayudó en su gran mayoría

Cuestión 12: Ejecute la consulta que pide los datos de la cuestión 9 de nuevo. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Comparar con los resultados anteriores.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
SELECT (COUNT(DISTINCT c.clienteid) * 100.0 / (SELECT COUNT(*) FROM clientes)) AS
porcentaje_clientes_andalucia FROM clientes c JOIN reservas r ON c.clienteid = r.clienteid_clientes
JOIN vehiculos v ON r.vehiculoid_vehiculos = v.vehiculoid JOIN plazas p ON r.plazaid_plazas =
p.plazaid JOIN pagos p2 ON r.reservaid = p2.reservaid_reservas LEFT JOIN incidencias i ON
r.reservaid = i.reservaid_reservas
WHERE c.provincia IN ('Almería', 'Cádiz', 'Córdoba', 'Granada', 'Huelva', 'Jaén', 'Málaga', 'Sevilla')
AND EXTRACT(MONTH FROM r.fechainicio) IN (6, 7, 8) AND p2.cantidad > 150 AND p.numero
< -4 AND (i.estado != 'cerrada' OR i.estado IS NULL) AND v.color != 'negro';
porcentaje_clientes_andalucia
-----
3.6201904761904762
Time: 41982,668 ms (00:41,983)
```

Resultado comando EXPLAIN

```
Aggregate (cost=879545.37..879545.39 rows=1 width=32)
  InitPlan 1 (returns $0)
    -> Aggregate (cost=62836.40..62836.41 rows=1 width=8)
      -> Index Only Scan using idx_clientes_provincia on clientes (cost=0.43..55336.40
rows=3000000 width=0)
        -> Sort (cost=816668.07..816688.51 rows=8178 width=4)
          Sort Key: c.clienteid
          -> Nested Loop (cost=60527.35..816136.60 rows=8178 width=4)
            -> Nested Loop (cost=60526.79..777521.28 rows=7179 width=8)
              -> Nested Loop Left Join (cost=60526.36..749939.17 rows=7712 width=12)
                Filter: ((i.estado <> 'cerrada'::text) OR (i.estado IS NULL))
                -> Hash Join (cost=60525.93..715551.57 rows=9637 width=12)
                  Hash Cond: (r.clienteid_clientes = c.clienteid)
                  -> Hash Join (cost=4983.34..657316.54 rows=160167 width=12)
                    Hash Cond: (r.plazaid_plazas = p.plazaid)
                    -> Seq Scan on reservas r (cost=0.00..651562.22 rows=293703 width=16)
                      Filter: (EXTRACT(month FROM fechainicio) = ANY ('{6,7,8}'::numeric[]))
                    -> Hash (cost=3620.00..3620.00 rows=109067 width=4)
                      -> Seq Scan on plazas p (cost=0.00..3620.00 rows=109067 width=4)
                        Filter: (numero < '-4'::integer)
                  -> Hash (cost=52580.35..52580.35 rows=180500 width=4)
                    -> Bitmap Heap Scan on clientes c (cost=2026.35..52580.35 rows=180500
width=4)
                      Recheck Cond: (provincia = ANY ('
{Almería,Cádiz,Córdoba,Granada,Huelva,Jaén,Málaga,Sevilla}'::text[]))
                      -> Bitmap Index Scan on idx_clientes_provincia (cost=0.00..1981.22
rows=180500 width=0)
                        Index Cond: (provincia = ANY ('
{Almería,Cádiz,Córdoba,Granada,Huelva,Jaén,Málaga,Sevilla}'::text[]))
                      -> Index Scan using incidencias_reservaid_idx on incidencias i (cost=0.43..3.56
rows=1 width=12)
                        Index Cond: (reservaid_reservas = r.reservaid)
                    -> Index Scan using pk_vehiculos on vehiculos v (cost=0.43..3.58 rows=1 width=4)
                      Index Cond: (vehiculoid = r.vehiculoid_vehiculos)
```

Comentarios y explicaciones.

PostgreSQL genera un plan basado en Hash Join y Nested Loop, priorizando filtros tempranos y aprovechando los índices ya creados. Aplica los filtros más restrictivos como provincia, número de plaza y color antes de realizar los joins, lo que reduce significativamente el tamaño de las relaciones intermedias. En lugar de escaneos secuenciales completos, el optimizador usa Bitmap Index Scan e Index Scan cuando es posible (con los índices que ya habíamos creado anteriormente) reduciendo el coste de la búsqueda y de los joins.

De todas formas, al haber menos registros causado por el borrado anterior, se puede ver que el coste total es inferior al coste que hubo anteriormente (asimismo con el porcentaje de clientes pertenecientes a andalucía)

Cuestión 13: ¿Qué optimización/mejoras de la BD propondría para mejorar los resultados de dicho plan sin modificar el código SQL de la consulta? ¿Por qué? **No implementarlo ahora.**

En cuanto a optimizaciones y mejoras de la base de datos, se podría emplear Vacuum para liberar el espacio no utilizado y actualizar las estadísticas de la base de datos, y así ayudar a que PostgreSQL tenga información actualizada sobre la distribución de datos y pueda elegir el plan de ejecución más eficiente. Se puede usar un vacuum_freeze_min_age reduciendo las veces que se actualiza el mapeado de visibilidad para que el vacuum funcione agresivamente (No lo haremos)

El capítulo 24 de la documentación dice que es efectivo crear índices correspondientes para mejorar y optimizar la consulta y reducir los escaneos secuenciales, además de usar un REINDEX para reconstruir los índices. REINDEX DATABASE telpark;

Por otro lado, se podría emplear un analyze para tener las estadísticas de las tablas actualizadas y que el planificador tome las decisiones más eficientes (por ejemplo si creamos índices nuevos que ayuden a las consultas).

Cuestión 14: Usando PostgreSQL, lleve a cabo las operaciones propuestas en la cuestión anterior y ejecute el plan de ejecución de la consulta que pide los datos de la cuestión 9. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Compare los resultados del plan de ejecución con los de los apartados anteriores. Coméntelos.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
SELECT (COUNT(DISTINCT c.clienteid) * 100.0 / (SELECT COUNT(*) FROM clientes)) AS
porcentaje_clientes_andalucia FROM clientes c JOIN reservas r ON c.clienteid = r.clienteid_clientes
JOIN vehiculos v ON r.vehiculoid_vehiculos = v.vehiculoid JOIN plazas p ON r.plazaid_plazas =
p.plazaid JOIN pagos p2 ON r.reservaid = p2.reservaid_reservas LEFT JOIN incidencias i ON
r.reservaid = i.reservaid_reservas
WHERE c.provincia IN ('Almería', 'Cádiz', 'Córdoba', 'Granada', 'Huelva', 'Jaén', 'Málaga', 'Sevilla')
AND EXTRACT(MONTH FROM r.fechainicio) IN (6, 7, 8) AND p2.cantidad > 150 AND p.numero
< -4 AND (i.estado != 'cerrada' OR i.estado IS NULL) AND v.color != 'negro';
porcentaje_clientes_andalucia
```

```
-----
          3.6201904761904762
(1 row)
```

Time: 15163,476 ms (00:15,163)

Resultado comando EXPLAIN

QUERY PLAN

Aggregate (cost=625844.06..625844.08 rows=1 width=32)
InitPlan 1 (returns \$0)
-> Aggregate (cost=59100.00..59100.01 rows=1 width=8)
-> Seq Scan on clientes (cost=0.00..53850.00 rows=2100000 width=0)
-> Sort (cost=566716.67..566730.36 rows=5476 width=4)
Sort Key: c.clienteid
-> Nested Loop (cost=38120.52..566376.64 rows=5476 width=4)
-> Nested Loop (cost=38120.09..543958.76 rows=6820 width=8)
-> Nested Loop Left Join (cost=38119.66..536458.20 rows=7313 width=12)
Filter: ((i.estado <> 'cerrada'::text) OR (i.estado IS NULL))
-> Nested Loop (cost=38119.23..528602.63 rows=9140 width=12)
-> Hash Join (cost=38118.81..520792.13 rows=16898 width=16)
Hash Cond: (r.clienteid_clientes = c.clienteid)
-> Seq Scan on reservas r (cost=0.00..481901.46 rows=294041 width=16)
Filter: (EXTRACT(month FROM fechainicio) = ANY ('{6,7,8}'::numeric[]))
-> Hash (cost=36610.31..36610.31 rows=120680 width=4)

Comentarios y explicaciones.

Hemos empleado un analyze para actualizar las estadísticas y mejorar el plan de ejecución completamente además de emplear un vacuum full para reorganizar físicamente las tablas y mejorar el rendimiento eliminando el espacio muerto. Por otro lado reconstruimos los índices de la database telpark con REINDEX DATABASE telpark;

Se nota en una diferencia de 30 segundos la gran optimización que hemos conseguido hacer. En cuanto al planificador. Gracias a explain.dalibo podemos ver que en cuanto al árbol anterior, en el hash join que se emplea en clientes y reservas se ha cambiado el plan de ejecución y ahora se emplean los bitmap index scan correctamente y se evita utilizar otro hash join sobre reservas y plazas.

Hablando de coste, se ve que se disminuye bastante en comparación con el anterior plan de ejecución. En conclusión, vemos que en el escaneo de clientes evita el scan completo y usa bitmap scan, además de el ajuste de los hash join, aprovechando a su vez todos los índices posibles (no como anteriormente) y reduciendo al mínimo el tiempo de ejecución.

El escaneo de clientes se ha cambiado por la reestructuración de los índices y la actualización de las estadísticas.

Cuestión 15: A partir de lo visto y recopilado en toda la práctica. Describir y comentar cómo es el proceso de procesamiento y optimización que realiza PostgreSQL en las consultas del usuario.

Cuando mandamos una consulta en PostgreSQL, no se ejecuta tal cual. Primero la analiza para ver si está bien escrita y genera una estructura interna. Luego pasa por la fase de Rewriting, donde puede cambiar algunas partes de la consulta si hay vistas o reglas aplicadas.

Después viene la parte más importante: el Planning y Optimization. Aquí PostgreSQL estudia varias formas posibles de ejecutar la consulta. Postgres estudia y aplica las reglas heurísticas, estudiando varios árboles de optimización y elige y calcula que será más rápido y barato, usando estadísticas de las tablas e índices. Decide si hacer un escaneo completo o usar índices, qué tipo de join aplicar, y en qué orden.

Con el plan elegido, empieza la ejecución del mismo, lee los datos, aplica filtros, hace joins y devuelve el resultado. Si puede, usa ejecución en paralelo para ir más rápido.

En la práctica comprobamos que crear índices adecuados, mantener las estadísticas actualizadas y ajustar parámetros de memoria mejora muchísimo el rendimiento. También vimos que usando EXPLAIN se puede analizar cómo PostgreSQL piensa ejecutar una consulta y detectar que partes se pueden optimizar.

Bibliografía

PostgreSQL (17)

- Capítulo 14: Performance Tips.
- Capítulo 19: Server Configuration.
- Capítulo 15: Parallel Query.
- Capítulo 24: Routine Database Maintenance Tasks.
- Capítulo 50: Overview of PostgreSQL Internals.
- Capítulo 68: How the Planner Uses Statistics.
- <https://pgtune.leopard.in.ua/>
- <https://explain.dalibo.com/>